

Paradigmas de Programação — Aula 08:

Concorrência, gorrotinas e canais

Go e gorrotinas

Go em uma tela: funções, variáveis e laço

```
package main

import "fmt"

func dobro(x int) int {           // tipo depois do nome
    return x * 2
}

func main() {
    var nome string = "Go"        // var com tipo explícito
    versao := 1.23                // := declara e infere
    total := 0
    for i := 1; i <= 10; i++ {    // for é o único laço
        total += i
    }
    fmt.Println(nome, versao, dobro(21), total)
}
```

Slice, struct e erro como valor

```
for _, n := range []int{3, 9, 4, 1} { ... } // range: índice e valor

type Fornecedor struct {
    Nome string
    Preço float64
}
f := Fornecedor{Nome: "Loja A", Preço: 99.90}

func dividir(a, b int) (int, error) { // dois retornos
    if b == 0 {
        return 0, errors.New("divisao por zero")
    }
    return a / b, nil
}
```

- Não há classes: `struct` é um tipo de dado
- Não há exceções: a falha volta como valor, e quem chama confere se `err` é `nil`

Gorrotina

- **Gorrotina** (*goroutine*): função executada de forma concorrente, criada com `go` antes da chamada

```
tarefa("A")    // espera tarefa terminar  
go tarefa("A") // segue imediatamente
```

- Escalonada pelo **runtime de Go**, não pelo sistema operacional
 - poucas threads do sistema (uma por núcleo), muitas gorrotinas sobre elas: escalonamento *M:N*
 - pilha inicial de poucos kilobytes, contra cerca de 1 MB de uma thread do sistema
 - milhares de gorrotinas num programa comum

O `main` não espera: `sync.WaitGroup`

```
var wg sync.WaitGroup
wg.Add(2) // duas tarefas pendentes
go func() { defer wg.Done(); tarefa("A") }()
go func() { defer wg.Done(); tarefa("B") }()
wg.Wait() // bloqueia até o contador zerar
```

`06-gorrotinas.go` — `tarefa` tem três passos de 100 ms:

Forma	Saída	Tempo
em sequência	A, A, A, B, B, B	600 ms
gorrotinas + <code>wait</code>	passos intercalados	300 ms
gorrotinas sem espera	só o primeiro passo de cada uma	o programa termina antes delas